

CS 112

Introduction to Data Structures

WEEK 04

PyList: Dynamic Arrays

Eric Araújo

Calvin University · Fall 2026

This Week

- 1 Why a plain array isn't enough
- 2 `new` and `delete`
- 3 The stack and the heap
- 4 Building a dynamic array
- 5 Growing when full
- 6 Destructors: now you need one
- 7 Copy constructors: shallow vs deep **KEY**

Arrays: The Good and the Bad

Good



- Simple to declare
- Indexing is fast: one multiply and add
- Contiguous, so the cache loves them

Bad



- Size fixed at **compile time**
- Can't grow when you need one more
- No idea how many slots are actually used

This week we will build a Python-like list that grows if needed.

Dynamic Memory Allocation

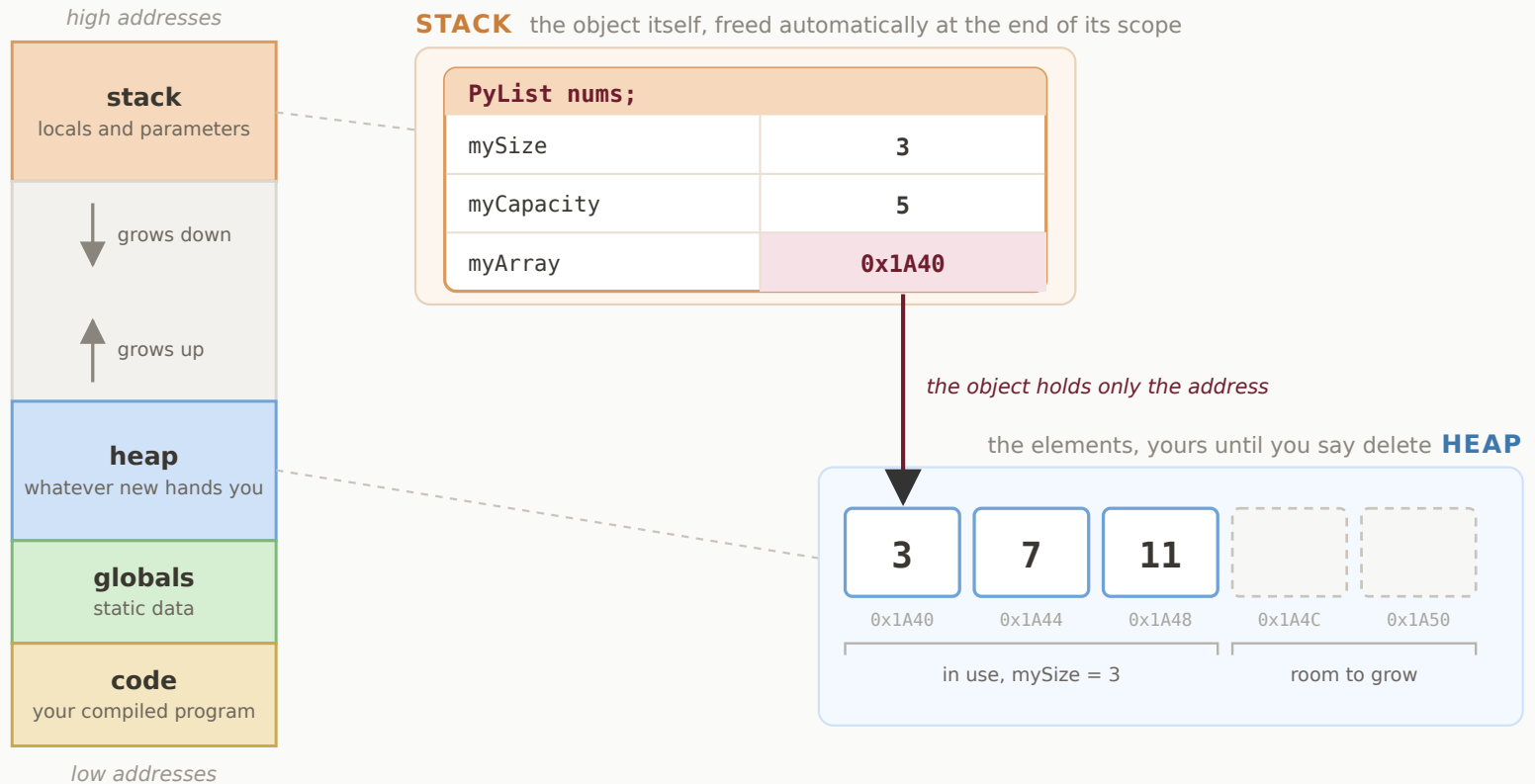
```
int *arr = new int[capacity];    // ask the OS for space
                                   // and a delete to match

arr[0] = 42;                      // use it like an array

delete [] arr;                   // free the space afterwards
arr = nullptr;                  // don't keep the address
```

- `new` returns the **address** of the block
- The size can be a variable: decided while the program runs
- `delete []` for arrays, plain `delete` for single objects

Stack and Heap



The object is on the stack. Its data is on the heap. Only the heap part needs a delete .

A Dynamic Array

```
class PyList {
public:
    PyList();                // starts empty, with no array
    PyList(const PyList &original); // copy: builds its own array
    ~PyList();               // frees the array
    unsigned getSize() const; // how many items are in it
    void append(const Item &it); // add one more at the end
    Item& operator[](unsigned i); // v[i], on either side of an =
private:
    Item *myArray;           // address of the block, or nullptr
    unsigned mySize;         // slots used so far
    unsigned myCapacity;     // slots the block has room for
};
```

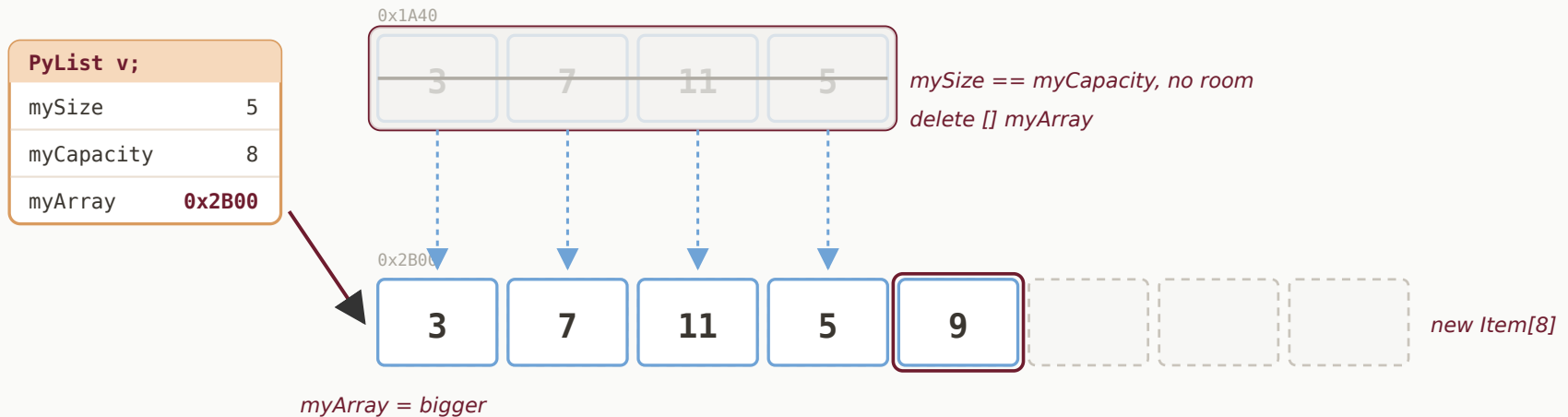
size is how many items you have. **capacity** is how many fit before it must grow.

Appending

```
void PyList::append(const Item &it) {  
    if (mySize == myCapacity) {           // no room left  
        myCapacity = (myCapacity == 0)  
            ? 1                               // first append  
            : myCapacity * 2;               // else double  
        Item *bigger = new Item[myCapacity];  
        for (unsigned i = 0; i < mySize; ++i) {  
            bigger[i] = myArray[i];         // copy each one  
        }  
        delete [] myArray;                 // free the old  
        myArray = bigger;                  // keep the new  
    }  
    myArray[mySize] = it;                  // room now  
    ++mySize;  
}
```

Most appends are two operations. A full array costs n copies first.

Growing, Step by Step



TALK TO YOUR NEIGHBOR · TTYN

Why double the capacity instead of adding one slot?

- A.** Doubling uses less memory
- B.** Adding one would make every append copy the whole array
- C.** The heap only hands out sizes that are powers of two
- D.** It doesn't matter, both are equally fast

Now You Need a Destructor

```
PyList::~~PyList() {  
    delete [] myArray;  
    myArray = nullptr;  
}
```

C++ writes a destructor for you, but it only destroys the *members*. It destroys the pointer, not what the pointer points at.

The rule: if your class calls `new`, your class must define a destructor. Otherwise every `PyList` that dies leaves its heap block behind.

TALK TO YOUR NEIGHBOR · TTYN

When does a destructor run?

- A.** Only when you write `delete`
- B.** When a local object reaches the end of its scope, or is deleted
- C.** At the end of `main()` , for every object
- D.** When the garbage collector decides

The Copy That Bites

```
PyList a;  
a.append(3);  
a.append(7);
```

```
PyList b = a;           // what exactly got copied?
```

a.myArray	0x5f2a10	→ [3 7]
b.myArray	0x5f2a10	→ the same block!

The default copy duplicates the **pointer**, not the data.
Two objects now own one block, a **shallow copy**.

TALK TO YOUR NEIGHBOR · TTYN

What goes wrong with that shallow copy?

- A.** Nothing: sharing the block saves memory
- B.** Changing `b[0]` also changes `a[0]`
- C.** Both destructors delete the same block, undefined behavior
- D.** Both B and C

Deep Copy: the Copy Constructor

```
PyList::PyList(const PyList &orig) {  
    mySize      = orig.mySize;  
    myCapacity  = orig.myCapacity;  
    myArray     = new Item[myCapacity];    // our OWN block  
    for (unsigned i = 0; i < mySize; ++i) {  
        myArray[i] = orig.myArray[i];      // copy the values  
    }  
}
```

Same shape as the growth code: allocate, copy, own. Now `a` and `b` are independent, and each destructor frees its own block.

Three Times It Copies

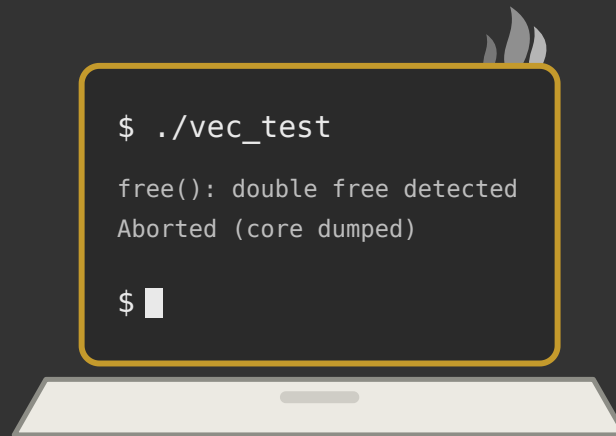
```
PyList b = a;                                // 1. explicit copy

void show(PyList v);                          // 2. pass-by-value
show(a);

PyList makeList() {                           // 3. return by value
    PyList local;
    return local;
}
```

Case 2 is why week 1 said *pass big objects by const reference*. Every by-value call to `show()` copies the whole heap block.

Demo



Watching a shallow copy destroy itself.

(two objects, one array, zero survivors)

Returning a Reference: `operator[]`

```
Item& PyList::operator[](unsigned i) {  
    return myArray[i];           // a reference, not a copy  
}  
  
v[2] = 99;           // works because v[2] IS the slot
```

Return `Item` and you get a copy, assigning to it would change a temporary and vanish. Returning `Item&` makes the call an *alias* for the element.

The `this` Pointer

Python

```
def set_first(self, v):  
    self.my_first = v
```

C++

```
void setFirst(Item v) {  
    this->myFirst = v;  
}
```

C++ passes the object silently as `this` : a pointer to the object the method was called on. You rarely write it, but it's there, and `this->x` is just `(*this).x`.

Week 04 Recap

- `new` gets heap memory at runtime; every `new` needs one `delete`
- **size** vs **capacity** is what makes append cheap most of the time
- Growing means allocate, copy, delete, repoint: linear, so we double
- If your class calls `new`, it needs a **destructor**
- The default copy is **shallow**: aliasing plus a double delete
- Write a copy constructor that allocates its own block

Next: a05 →

